

```

1  import java.awt.Cursor;
10
11 /**
12  * View class.
13  *
14  * @author Eric
15  */
16 @SuppressWarnings("serial")
17 public final class AppendUndoView1 extends JFrame implements AppendUndoView {
18
19     /**
20      * Controller object.
21      */
22     private AppendUndoController controller;
23
24     /**
25      * GUI widgets that need to be in scope in actionPerformed method, and
26      * related constants. (Each should have its own Javadoc comment, but these
27      * are elided here to keep the code shorter.)
28      */
29     private static final int LINES_IN_TEXT_AREAS = 5, LINE_LENGTHS_IN_TEXT_AREAS = 20,
30         ROWS_IN_BUTTON_PANEL_GRID = 1, COLUMNS_IN_BUTTON_PANEL_GRID = 2,
31         ROWS_IN_THIS_GRID = 3, COLUMNS_IN_THIS_GRID = 1;
32
33     /**
34      * Text areas.
35      */
36     private final JTextArea inputText, outputText;
37
38     /**
39      * Buttons.
40      */
41     private final JButton resetButton, appendButton, undoButton;
42
43     /**
44      * No-argument constructor.
45      */
46     public AppendUndoView1() {
47         // Create the JFrame being extended
48
49         /*
50          * Call the JFrame (superclass) constructor with a String parameter to
51          * name the window in its title bar
52          */
53         super("Simple GUI Demo With MVC");
54
55         // Set up the GUI widgets -----
56
57         /*
58          * Create widgets
59          */
60         this.inputText = new JTextArea("", LINES_IN_TEXT_AREAS,
61             LINE_LENGTHS_IN_TEXT_AREAS);
62         this.outputText = new JTextArea("", LINES_IN_TEXT_AREAS,
63             LINE_LENGTHS_IN_TEXT_AREAS);
64         this.resetButton = new JButton("Reset");
65         this.appendButton = new JButton("Append Input");
66         this.undoButton = new JButton("Undo");
67         /*
68          * Text areas should wrap lines, and outputText should be read-only
69          */
70         this.inputText.setEditable(true);
71         this.inputText.setLineWrap(true);
72         this.inputText.setWrapStyleWord(true);
73         this.outputText.setEditable(false);

```

```

74  ... ..
75  this.outputText.setLineWrap(true);
76  this.outputText.setWrapStyleWord(true);
77  /*
78   * Create scroll panes for the text areas in case text is long enough to
79   * require scrolling in one or both dimensions
80   */
81  JScrollPane inputTextScrollPane = new JScrollPane(this.inputText);
82  JScrollPane outputTextScrollPane = new JScrollPane(this.outputText);
83  /*
84   * Create a button panel organized using grid layout
85   */
86  JPanel buttonPanel = new JPanel(
87      new GridLayout(ROWS_IN_BUTTON_PANEL_GRID, COLUMNS_IN_BUTTON_PANEL_GRID));
88  /*
89   * Add the buttons to the button panel, from left to right and top to
90   * bottom
91   */
92  buttonPanel.add(this.resetButton);
93  buttonPanel.add(this.appendButton);
94  buttonPanel.add(this.undoButton);
95  /*
96   * Organize main window using grid layout
97   */
98  this.setLayout(new GridLayout(ROWS_IN_THIS_GRID, COLUMNS_IN_THIS_GRID));
99  /*
100   * Add scroll panes and button panel to main window, from left to right
101   * and top to bottom
102   */
103  this.add(inputTextScrollPane);
104  this.add(buttonPanel);
105  this.add(outputTextScrollPane);
106
107  // Set up the observers -----
108
109  /*
110   * Register this object as the observer for all GUI events
111   */
112  this.resetButton.addActionListener(this);
113  this.appendButton.addActionListener(this);
114  this.undoButton.addActionListener(this);
115
116  // Start the main application window -----
117
118  /*
119   * Make sure the main window is appropriately sized for the widgets in
120   * it, that it exits this program when closed, and that it becomes
121   * visible to the user now
122   */
123  this.pack();
124  this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
125  this.setVisible(true);
126
127  }
128
129  /**
130   * Register argument as observer/listener of this; this must be done first,
131   * before any other methods of this class are called.
132   *
133   * @param controller
134   *       controller to register
135   */
136  @Override
137  public void registerObserver(AppendUndoController controller) {
138      this.controller = controller;
139  }

```

139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197 }
198

```
/**
 * Updates input display based on String provided as argument.
 *
 * @param input
 *      new value of input display
 */
@Override
public void updateInputDisplay(String input) {
    this.inputText.setText(input);
}

/**
 * Updates output display based on String provided as argument.
 *
 * @param output
 *      new value of output display
 */
@Override
public void updateOutputDisplay(String output) {
    this.outputText.setText(output);
}

@Override
public void updateUndoAllowed(boolean allowed) {
    this.undoButton.setEnabled(allowed);
}

@Override
public void actionPerformed(ActionEvent event) {
    /*
     * Set cursor to indicate computation on-going; this matters only if
     * processing the event might take a noticeable amount of time as seen
     * by the user
     */
    this.setCursor(Cursor.getPredefinedCursor(Cursor.WAIT_CURSOR));
    /*
     * Determine which event has occurred that we are being notified of by
     * this callback; in this case, the source of the event (i.e, the widget
     * calling actionPerformed) is all we need because only buttons are
     * involved here, so the event must be a button press; in each case,
     * tell the controller to do whatever is needed to update the model and
     * to refresh the view
     */
    Object source = event.getSource();
    if (source == this.resetButton) {
        this.controller.processResetEvent();
    } else if (source == this.appendButton) {
        this.controller.processAppendEvent(this.inputText.getText());
    } else if (source == this.undoButton) {
        this.controller.processUndoEvent();
    }
    /*
     * Set the cursor back to normal (because we changed it at the beginning
     * of the method body)
     */
    this.setCursor(Cursor.getDefaultCursor());
}
```